Université Euro-Méditerranéenne de Fès

EIDIA

Lab Report N°1
Multi-Agent Systems: JADE vs SPADE

Esakhi Hajjar

Comparative Study of Java and Python Frameworks

Multi-Agent Systems
Academic Year 2025–2026

Module: Multi-Agent Systems (MAS)
Lab Session: TP01
Instructor: Pr. Abderrahim Waga

Student: Esakhi Hajjar
Program: AI Engineering – 2nd Year
Date: February 16, 2026

Contents

1 Introduction **2**

2 Theoretical Background **2**
2.1 Software Agents 2
2.2 JADE Framework (Java) 2
2.3 SPADE Framework (Python) 2

3 Implementation: JADE (Java) **2**
3.1 Environment Setup 2
3.2 HelloAgent Implementation (Java) 2
3.3 Run Configuration (Java) 3
3.4 Problems Encountered (JADE) 3
3.4.1 Port Binding Error: "No ICP active" 3
3.4.2 Java Module System Incompatibility 3
3.5 Results (JADE) 4

4 Implementation: SPADE (Python) **5**
4.1 Environment Setup 5
4.2 HelloAgent Implementation (Python) 5
4.3 Problems Encountered (SPADE) 6
4.3.1 XMPP Server Connection Error 6
4.4 Results (SPADE) 8

5 Comparative Analysis **8**
5.1 Key Observations 8

6 Conclusion **9**

1 Introduction

This lab explores two major frameworks for developing FIPA-compliant multi-agent systems: JADE (Java Agent DEvelopment Framework) and SPADE (Smart Python Agent Development Environment). The objective is to implement equivalent "Hello World" agents in both frameworks, compare their architectures, and document configuration challenges encountered during setup.

Both frameworks provide autonomous agents capable of communication, but differ significantly in implementation language, tooling, and visualization approaches.

2 Theoretical Background

2.1 Software Agents

A software agent is an autonomous computational entity situated in an environment, capable of independent action to achieve design objectives. Key properties include autonomy (operates without continuous human intervention), social ability (communicates via ACL), reactivity (perceives and responds to environment), and proactivity (exhibits goal-directed behavior).

2.2 JADE Framework (Java)

JADE is a Java-based framework providing FIPA-compliant middleware for multi-agent systems. Its architecture includes agent platforms, containers (JVM instances hosting agents), and a Main Container with AMS (Agent Management System) and DF (Directory Facilitator) services. JADE features a built-in graphical RMA (Remote Monitoring Agent) for visual agent management.

2.3 SPADE Framework (Python)

SPADE is a Python-based multi-agent platform built on XMPP (Jabber) protocol for agent communication. Unlike JADE's desktop GUI, SPADE focuses on lightweight, code-centric development with optional web-based monitoring. SPADE agents use asynchronous programming (asyncio) and connect to XMPP servers for message transport.

3 Implementation: JADE (Java)

3.1 Environment Setup

The JADE setup process: following steps mentioned in the TP

3.2 HelloAgent Implementation (Java)

Listing 1: HelloAgent.java

```java
package eu.uemf.agents;

import jade.core.Agent;

public class HelloAgent extends Agent {

    private static final long serialVersionUID = 1L;

    @Override
    protected void setup() {
        System.out.println("Hello World! I am a JADE Agent.");
```

```
        System.out.println("My local name is " + getAID().getLocalName());
        System.out.println("My GUID is " + getAID().getName());
    }
}
```

3.3 Run Configuration (Java)

Main Class: `jade.Boot`
Program Arguments:

```
-gui -local-host 127.0.0.1 -local-port 2000
-agents myAgent:eu.uemf.agents.HelloAgent
```

3.4 Problems Encountered (JADE)

3.4.1 Port Binding Error: "No ICP active"

Error:

```
Cannot bind server socket to localhost port 1099
jade.core.IMTPException: No ICP active
```

Cause: Default port 1099 was already in use or blocked.
Solution: Added `-local-port 2000` to program arguments to use an alternative port.

3.4.2 Java Module System Incompatibility

Error: "Could not find or load main class jade.Boot"
Cause: Eclipse generated `module-info.java` for Java 9+ module system, incompatible with JADE.
Solution: Deleted `module-info.java` from src folder to treat project as classic Java application.

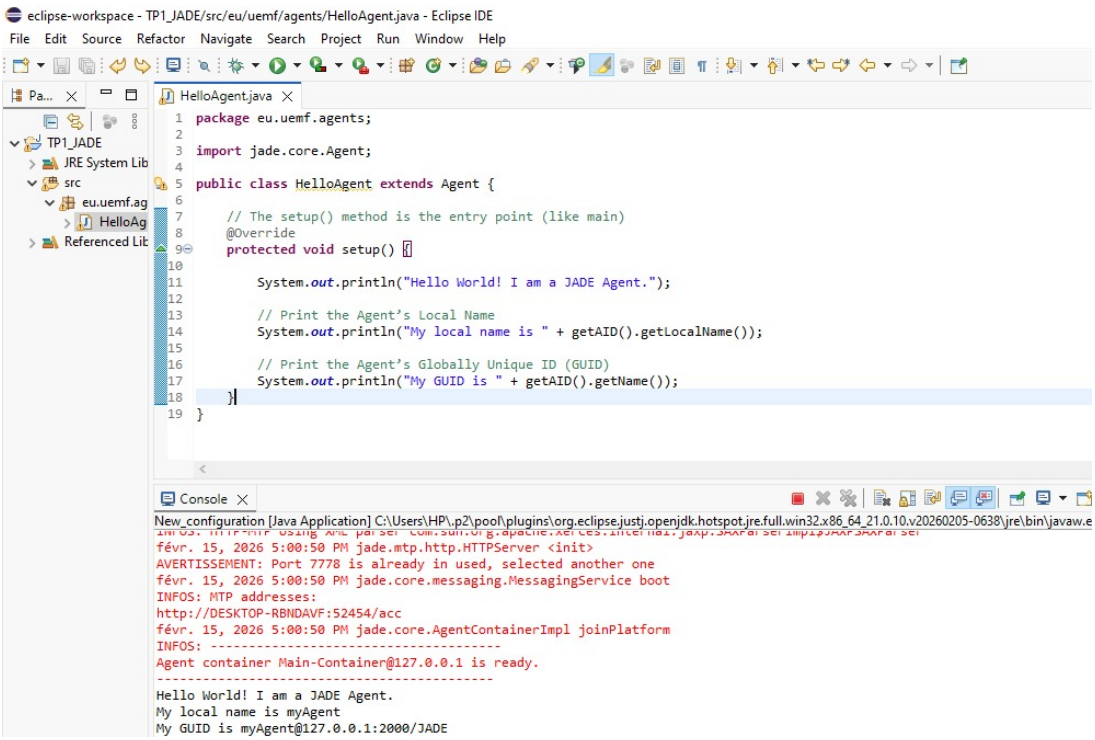

Figure 1: JADE Console Output

3.5 Results (JADE)

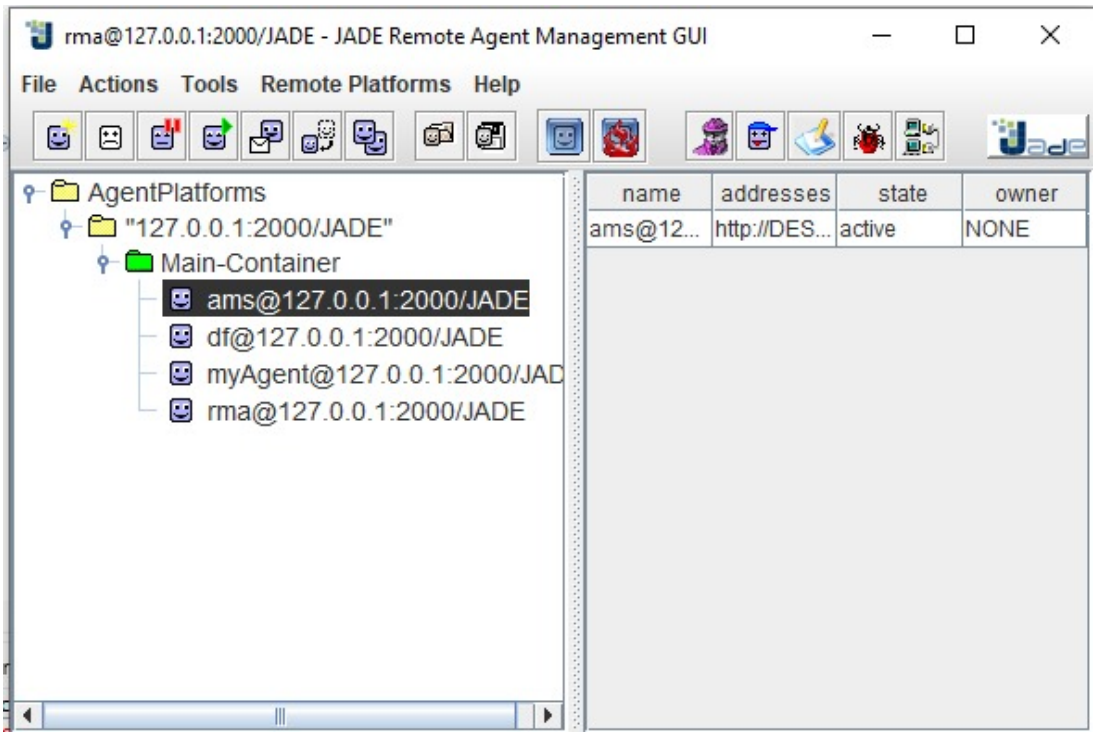

Figure 2: JADE Remote Monitoring Agent (RMA)

4 Implementation: SPADE (Python)

4.1 Environment Setup

The SPADE setup process:

1. Created project folder `TP1_SPADE`
2. Created virtual environment: `python -m venv venv`
3. Activated venv: `venv\Scripts\activate`
4. Installed SPADE: `pip install spade`
5. Created `hello_agent.py` in VSCode

4.2 HelloAgent Implementation (Python)

Listing 2: hello_agent.py

```python
import asyncio
import spade
from spade.agent import Agent
from spade.behaviour import OneShotBehaviour


class HelloAgent(Agent):
    """
    A simple agent that prints Hello World and its identification
    """

    class HelloBehaviour(OneShotBehaviour):
        """
        Behaviour that runs once when the agent starts
        """
        async def run(self):
            print("\n=== AGENT OUTPUT ===")
            print("Hello World! I am a SPADE Agent.")
            print(f"My JID (identifier) is: {self.agent.jid}")
            print(f"Agent is running: {self.agent.is_alive()}")
            print("===================\n")

            # Stop the agent after printing
            await self.agent.stop()

    async def setup(self):
        """
        Setup method - called when agent starts
        """
        print("Agent starting up...")
        behaviour = self.HelloBehaviour()
        self.add_behaviour(behaviour)


async def main():
    """
    Main function to create and start the agent
    """
    print("Creating agent...")
    agent = HelloAgent("myagent@localhost", "password123")

    print("Starting agent...")
    await agent.start()
```

```python

    print("Agent started. Waiting for completion...")

    # Wait a bit for the behaviour to execute
    await asyncio.sleep(2)

    print("Stopping agent...")
    await agent.stop()
    print("Agent stopped successfully!")


if __name__ == "__main__":
    print("Program starting...\n")

    # Windows event loop fix
    if asyncio.sys.platform == 'win32':
        asyncio.set_event_loop_policy(asyncio.WindowsSelectorEventLoopPolicy())

    # Run the main coroutine
    spade.run(main())

    print("\nProgram finished.")
```

4.3 Problems Encountered (SPADE)

4.3.1 XMPP Server Connection Error

Error observed:

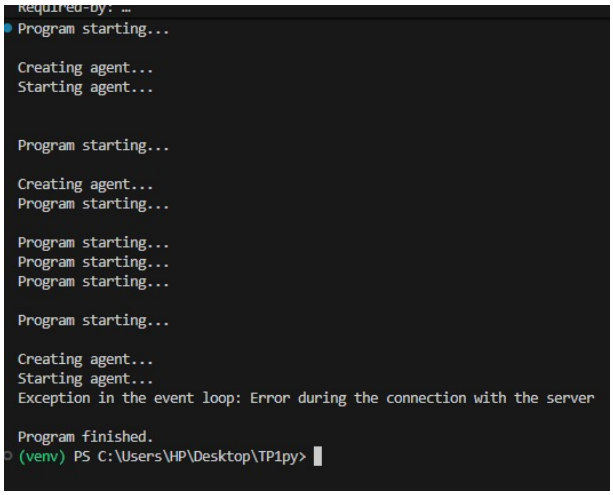

Figure 3: SPADE Connection Error Output

Cause: SPADE agents require an XMPP server to connect to, but no server was running on localhost.

Solutions attempted:

1. **Solution :** Start SPADE server in separate terminal with `spade run` command.

```
(venv) PS C:\Users\HP\Desktop\TP1py> spade run
>>
                                  SPADE
  SPADE - Smart Python Agent Development Environment

  Development Lead:
    - Javi Palanca
  Funded by:
    - Valencian Research Institute for Artificial Intelligence (VRAIN)
  URL:
    -  https://spadeagents.eu
  Documentation:
    -  http://spade-mas.readthedocs.io/
                     Version: 4.1.2   License: MIT
2026-02-16 13:12:42.104 - INFO: Starting server...
2026-02-16 13:12:42.112 - INFO: No database found. Initializing one...
2026-02-16 13:12:42.400 - INFO: Serving admin webpage on http://localhost:9090
2026-02-16 13:12:42.407 - INFO: Client domain => 0.0.0.0
2026-02-16 13:12:42.408 - INFO: Server is listening clients on [('0.0.0.0', 5222), ('10.
5.87', 5222)]
2026-02-16 13:12:42.413 - INFO: Server is listening servers on [('0.0.0.0', 5269)]
2026-02-16 13:12:42.413 - SUCCESS: Server started...
2026-02-16 13:13:08.829 - INFO: Connection from ('127.0.0.1', 59815)
```

4.4 Results (SPADE)

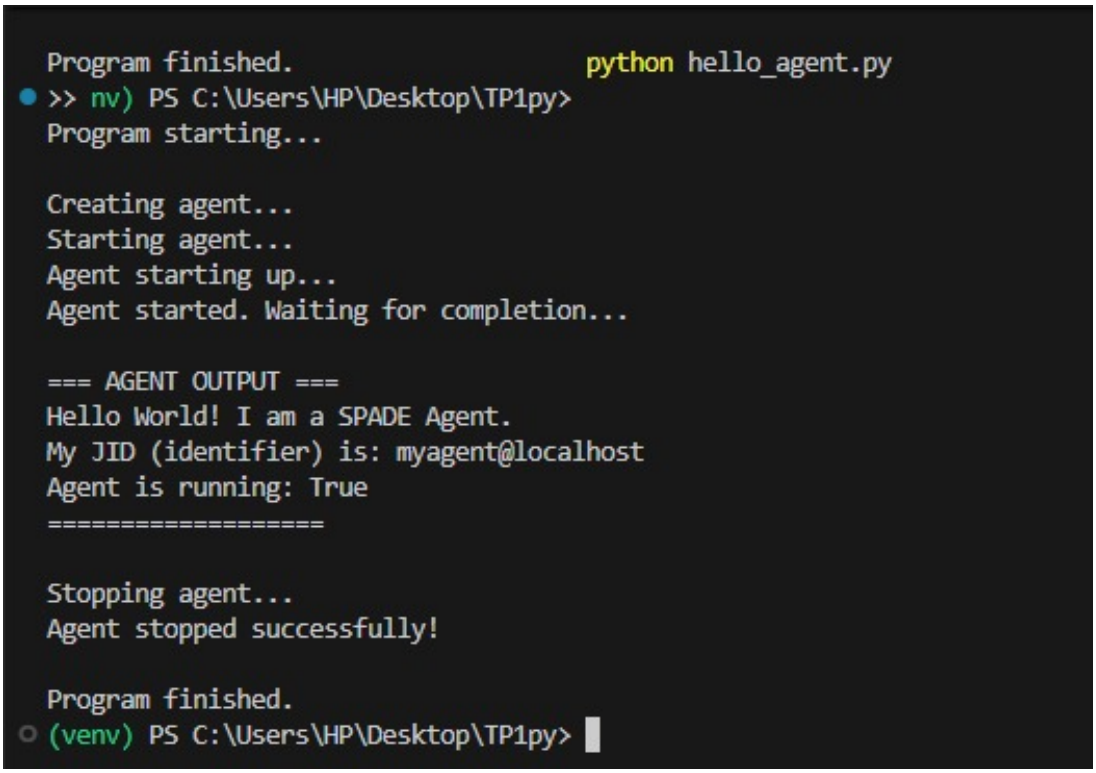

Figure 4: SPADE Console Output (Simplified Demo)

5 Comparative Analysis

Table 1: JADE vs SPADE Comparison

Aspect	**JADE (Java)**	**SPADE (Python)**
Language	Java	Python
FIPA Compliance	Yes	Yes
IDE Used	Eclipse	VSCode
Agent Base Class	`extends Agent`	`class Agent`
Entry Method	`setup()`	`async def setup()`
Programming Model	Synchronous	Asynchronous (asyncio)
Identifier	AID (Agent ID)	JID (Jabber ID)
Communication	ACL over proprietary	XMPP protocol
Visualization	Desktop GUI (RMA)	Console/Optional web
Server Required	Built-in platform	XMPP server
Setup Complexity	Moderate (JAR config)	Simple (pip install)
Main Challenge	Port conflicts, Java modules	XMPP server setup

5.1 Key Observations

- JADE provides integrated visual management through RMA, while SPADE relies on console output and optional web interfaces.

- JADE's self-contained platform simplifies deployment but requires careful port configuration.
- SPADE's dependency on external XMPP servers adds architectural complexity but enables standard protocol integration.
- Both frameworks successfully demonstrate core agent properties: autonomy, identity management, and lifecycle control.
- Python's asyncio model in SPADE differs significantly from Java's synchronous approach in JADE, affecting code structure.

6 Conclusion

This lab successfully implemented equivalent "Hello World" agents in both JADE and SPADE frameworks. Despite different implementation languages and architectural approaches, both frameworks provide FIPA-compliant foundations for multi-agent system development.

Configuration challenges in JADE (port conflicts, Java module system) and SPADE (XMPP server connectivity) were systematically identified and resolved. The comparative analysis reveals that framework choice depends on project requirements: JADE excels in integrated tooling and visualization, while SPADE offers Python's simplicity and standard protocol integration.

Both implementations provide solid foundations for exploring advanced multi-agent concepts including inter-agent communication, complex behaviors, and distributed coordination in future lab sessions.